Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier

The Resourcerer Supreme: A Deep Reinforcement Learning Agent for Real-Time Scalability on Stateless Cloud Applications

CONSTANTINOS BITSAKOS[1], DIMITRIOS TSOUMAKOS[2], IOANNIS KONSTANTINOU[3], NECTARIOS KOZIRIS[4]

[1]CSLAB, NTUA, Athens, Greece (e-mail: kbitsak@cslab.ece.ntua.gr)
[2]DBLAB, NTUA, Athens, Greece (e-mail: dtsouma@mail.ntua.gr)
[3]UNIVERSITY OF THESSALY, Greece (e-mail: ikons@uth.gr)
[4]CSLAB, NTUA, Athens, Greece (e-mail: nkoziris@cslab.ece.ntua.gr)

Corresponding author: Constantinos Bitsakos (e-mail: kbitsak@cslab.ece.ntua.gr).

ABSTRACT Kubernetes (K8s) has become the de facto standard for running containerized applications. The issue is that built-in autoscaling solutions, like the Horizontal Pod Autoscaler (HPA) struggle with highly dynamic workloads, particularly in fast, in-memory data stores such as Redis, where sudden demand surges require intelligent, proactive scaling rather than just reactive adjustments.

In this paper, we introduce the Resourcerer Supreme (RS), a Deep Reinforcement Learning (Deep RL)-driven autoscaling controller that dynamically adjusts the size of a K8s-Redis cluster based on learned workload patterns. RS leverages LSTM layers to forecast load trends and proactively scale resources before traffic spikes occur, ensuring a more stable and efficient system. Built upon our previous system, DERP, RS surpasses it by 40% in simulation tests focused on Redis workloads in terms of accumulated reward while eliminating the negative reward patterns observed in HPA.

We further introduce RS Transfer, an enhanced controller that uses transfer learning to generalize across different environments and applications. By leveraging pre-trained policies, RS Transfer not only outperforms RS by **over an order of magnitude** in simulations but also delivers a **5.5× performance boost** in real-world Kubernetes deployments, demonstrating its ability to rapidly adapt with minimal retraining.

Both RS and RS Transfer act as intelligent scaling controllers, dynamically resizing the Redis cluster based on Deep RL-driven decisions. Our custom reward function balances throughput, pod count, CPU usage, memory overhead and latency, leading to a proactive and highly efficient autoscaling approach. These results highlight the advantages of predictive Deep RL-based controllers in cloud-native environments, reducing manual tuning and ensuring robust performance even under unpredictable workload fluctuations.

INDEX TERMS Kubernetes, Deep Reinforcement Learning, LSTMs, Autoscaling, Redis, Transfer Learning, Controllers

I. INTRODUCTION

Kubernetes (K8s) has emerged as a leading platform for orchestrating containerized applications in cloud environments, offering robust deployment, scaling, and lifecycle management [1]. Redis, an in-memory key-value data store, is widely used in both stateful and stateless configurations depending on the application requirements. In this work, we adopt Redis in its stateless mode—by disabling persistence and replication—which aligns well with our focus on real-time autoscaling and simplifies deployment within Kubernetes. Redis integrates seamlessly with Kubernetes due to its high-speed data access and flexibility [2]. Ensuring efficient scalability in such deployments is essential to handle fluctuating workloads, as any latency or resource imbalance can degrade performance in distributed systems [3]. In practice, dynamically balancing compute, memory, and network utilization is crucial for real-time responsiveness to varying demands.

Redis is widely adopted by industry leaders such as

Twitter, Airbnb, Tinder, Yahoo, Adobe, Hulu, Amazon, and OpenAI. Its ability to rapidly cache and retrieve data makes it indispensable in applications that require high-speed access and low latency, which is vital for handling real-time workloads. In our work, Redis forms the backbone of the autoscaling experiments, where our Resourcerer Supreme (RS) controllers leverage real-time metrics from Redis to make both proactive and reactive scaling decisions, thereby enhancing resource efficiency and ensuring consistent performance under dynamic load conditions.

Redis is widely adopted by industry leaders such as Twitter, Airbnb, Tinder, Yahoo, Adobe, Hulu, Amazon, and OpenAI. Its ability to rapidly cache and retrieve data makes it indispensable in applications that require high-speed access and low latency, which is vital for handling real-time workloads. In our work, Redis forms the backbone of the autoscaling experiments, where our Resourcerer Supreme (RS) controllers leverage real-time metrics from Redis to make both proactive and reactive scaling decisions, thereby enhancing resource efficiency and ensuring consistent performance under dynamic load conditions.

Although Kubernetes includes the Horizontal Pod Autoscaler (HPA) for scaling pods based on CPU and memory usage [1], its reactive design and reliance on limited metrics often lead to over-provisioning and elevated latencies—particularly under rapidly changing or spiky workloads [4]. By contrast, Deep Reinforcement Learning (RL) has shown promise for resource optimization in cloud environments [5]. A key example is our previous work DERP [4], which employed Double Deep Q-learning for elastic resource provisioning in Cassandra clusters. DERP demonstrated that leveraging advanced Deep Reinforcement Learning techniques—transcending traditional Q-tables, Markov Chains, and decision-tree methods—can effectively manage complex, high-dimensional state spaces and achieve up to a 1.6× improvement in lifetime rewards. In these experiments, the reward function was custom-designed by the user or vendor to penalize overprovisioning, minimize costs, and ensure optimal resource utilization, thereby guaranteeing low latency and high throughput for user queries; further details of this tailored reward design are discussed in the subsequent chapters. This success underscores how deep RL can learn scalable autoscaling policies that reduce overhead and accommodate diverse workloads.

Building upon these insights, we introduce three key advancements. First, we propose an enhanced version of DERP tailored for Kubernetes and Redis, incorporating additional metrics and improved learning capabilities. Second, we present **Resourcerer Supreme (RS)**, a system that integrates Long Short-Term Memory (LSTM) layers [6] to predict workload spikes preemptively rather than reacting solely to current loads. Our custom reward function explicitly minimizes CPU, memory, pod count, and latency while maximizing throughput, enabling more proactive and effective scaling decisions.

Furthermore, we introduce an enhanced variant, RS Transfer, which utilizes transfer learning to significantly elevate performance and adaptability across diverse workloads, applications, and environments. Our simulation results indicate that while HPA consistently incurs negative rewards and DERP scores around 200K, RS attains between 240K and 310K—a roughly 40% improvement over DERP—whereas RS Transfer achieves rewards between 3.05M and 3.80M, exceeding RS by more than an order of magnitude. In real-world Kubernetes/Redis deployments, RS and RS Transfer deliver total rewards of 56,000 and 308,000 respectively (a 5.5× gain), all achieved via the aforementioned user-customized reward function, which is designed to avoid over-provisioning, reduce operational costs, and optimize resource usage while ensuring high throughput, minimal latency, and more robust handling of unexpected load spikes.

This paper thus demonstrates how advanced Deep RL methods, combined with LSTM architectures and transfer learning, can substantially outperform reactive autoscalers (like HPA and DERP) in both simulation and real-world scenarios. Beyond improving operational costs, RS and RS Transfer also lessen the manual intervention typically required to tune cloud resource allocation — an especially important benefit in large, distributed environments. Notably, RS Transfer enables rapid adaptation to new applications, deployment environments, and orchestration frameworks with minimal retraining, making it a versatile and scalable solution. We posit that such a proactive, adaptive approach to autoscaling holds considerable promise for broader Kubernetes deployments, including those spanning multiple services and heterogeneous workloads.

II. LITERATURE REVIEW

A growing body of literature recognizes Kubernetes (K8s) as a leading orchestration solution, which has driven the innovation of advanced autoscaling techniques. Robust scalability is achieved through mechanisms such as the Horizontal Pod Autoscaler (HPA) [7], Vertical Pod Autoscaler (VPA) [8], and Cluster Autoscaler [9]. HPA scales the number of pods horizontally based on observed CPU or memory usage [7]. VPA adjusts pod resource requests (CPU/memory) to match their real usage over time [8]. Cluster Autoscaler increases or decreases the number of worker nodes in the cluster based on pod scheduling status and resource availability [9]. These autoscaling mechanisms represent the standard scalability controllers in Kubernetes and are widely used across cloud-native deployments. In our work, we adopt them as baselines for comparison to showcase the benefits and improvements enabled by our RL-based approaches.

Among these, HPA is most widely adopted, dynamically adjusting the number of pods based on CPU and memory usage metrics. However, its reactive nature often proves inadequate for handling highly dynamic or spiky workloads, resulting in over-provisioning and latency issues [1]. In contrast, our approach leverages Deep Reinforcement Learning (DRL) combined with Long Short-Term Memory (LSTM) layers to anticipate future workload changes and enable

proactive scaling decisions.

Several studies have explored improving Kubernetes scalability using machine learning (ML) and reinforcement learning (RL). DERP (Deep Elastic Resource Provisioning) is a notable system that employs Deep RL by combining Q-learning [10] with neural networks, thereby overcoming the limitations of earlier RL-based autoscalers that relied on fixed-size Q-tables or decision trees [11], [12]. Importantly, DERP was originally applied to Cassandra clusters rather than to Kubernetes environments. While Q-learning is a foundational RL technique for optimizing decision-making in stochastic environments [13], previous systems such as Tiramola [14] and VCONF [15]—which applied RL and heuristic-based approaches—were less effective at capturing complex, long-term workload dependencies. DERP's use of deep RL allowed for more fine-grained, policy-driven autoscaling; yet, it still struggles with pronounced temporal dependencies. Our work outperforms DERP by presenting a Double Deep Q learning network that integrates LSTM layers [6], facilitating early detection of workload spikes and preemptive resource allocation in Kubernetes/Redis environments.

Beyond DERP, several ML-based autoscalers have been proposed. Imdoukh et al. [16] introduced an autoscaling system for containerized applications that employs LSTM networks to predict workload fluctuations, while Toka et al. [17] developed a framework targeting predictive resource allocation for Kubernetes edge clusters. Koukis et al. [18] further evaluated the impact of network overhead on autoscaling performance in constrained environments. In contrast, our Resourcerer Supreme (RS) model leverages advanced Deep Reinforcement Learning techniques integrated with LSTM architectures and transfer learning to capture complex temporal dependencies. By using deep neural networks, RS can effectively model and predict workload spikes, enabling proactive rather than reactive scaling. This approach significantly improves resource efficiency, reduces latency, and adapts seamlessly to varying workloads, thereby outperforming existing ML-based autoscaling methods.

Additional research has addressed Kubernetes scalability from various perspectives. Thurgood and Lennon [19] investigated worker-node autoscaling under dynamic workloads, and Lankes et al. [20] introduced HermitCore for extreme-scale computing. Within the Redis ecosystem, Kimm et al. [21] proposed SCADIS for reliable scalability, and Zhu et al. [22] presented Harmonia for near-linear scaling with in-network conflict detection. Ben Seghier and Kazar [23], along with other works [24], [25], have compared various storage and scaling approaches. Complementary studies on dynamic scaling for web services [26] and competitive scalability for cloud architectures [27] further highlight the multifaceted challenges of cloud scalability.

Research on LSTM network efficiency has also advanced, with Kuchaiev et al. [28] introducing parameter factorization strategies to reduce computational overhead and Yu et al. [6] providing comprehensive reviews of LSTM variants. Concurrently, studies such as those by Ikemoto et al. [29] and Yu et al. [5] have examined the applications of DRL to control problems and decision-making in dynamic environments. Although these contributions offer valuable insights into DRL methodologies, they do not specifically tackle the real-time autoscaling challenges in Kubernetes. Our RS models extend these concepts by embedding LSTM-based forecasting within a DRL framework, thereby enhancing autoscaling efficiency under dynamic workloads.

Kubernetes-based management frameworks such as Scanflow-K8s [30] and performance evaluations by Fogli et al. [31] underscore the ongoing challenges in dynamically scaling cloud workloads. A particularly compelling avenue is transfer learning, which has been shown to reduce training times and improve model generalization across diverse tasks [32]. By transferring knowledge from one domain to another, agents can rapidly adapt to new workload patterns and hardware settings—an essential capability in heterogeneous cloud environments. Our enhanced variant, **RS Transfer**, exploits transfer learning to achieve simulation rewards over an order of magnitude higher than RS and a 5.5-fold improvement in real-world tests, underscoring its practical superiority.

Recent work by Dang-Quang and Yoo [33] proposes a Bi-LSTM-based autoscaler for Kubernetes that employs a proactive approach based on the Monitor–Analyze–Plan–Execute (MAPE) loop. Their system leverages Bidirectional LSTM to forecast HTTP workload patterns and implements a cooling-down period to mitigate oscillations, achieving significantly faster prediction speeds and improved resource provision accuracy compared to both traditional LSTM and ARIMA-based models. While their proactive strategy enhances autoscaling performance over the default HPA, our approach goes further by integrating DRL to dynamically learn optimal scaling policies based on multiple system metrics (CPU, memory, latency). Moreover, by incorporating transfer learning, our method adapts more effectively across diverse workloads, yielding superior efficiency and accumulated rewards.

In summary, our Deep RL-based approaches address the key limitations of existing autoscaling systems by combining LSTM-based temporal prediction with transfer learning. This integration not only improves resource efficiency and system performance in Kubernetes/Redis environments but also establishes a more adaptive and robust framework for real-time autoscaling, capable of seamlessly operating across a variety of different environments and applications.

III. SYSTEM MODEL

A. KUBERNETES AND REDIS SETUP

In our setup, we utilize a Kubernetes (K8s) Figure 1 cluster to orchestrate a stateless Redis environment. Redis, being an in-memory key-value data store, fits well into such a stateless environment, which allows for simpler autoscaling mechanisms and fault tolerance. Kubernetes' native capability to manage containerized workloads enables the dynamic scaling of Redis pods based on incoming requests and system load, ensuring high availability and low latency. This setup

forms the foundation upon which our deep reinforcement learning agents operate to optimize resource scaling (see Figure 1.

Additionally, we employed Prometheus [34] and Grafana [35] for visualizing the results of our experiments. These tools provided us with real-time monitoring and insights into the system's performance, giving a clear understanding of how the system behaved under different workloads. However, it is important to note that we did not use Prometheus or Grafana for collecting metrics in our experiments; instead, we retrieved all metrics directly from the Kubernetes metrics server [36], which served as the source for our Deep RL agent's decision-making process Figure 1.

B. DEEP REINFORCEMENT LEARNING AGENTS

We designed and compared three distinct deep reinforcement learning agents within the K8s environment: one using dense layers, one using Long Short-Term Memory (LSTM) layers and one using LSTM layers combined with transfer learning logic. The agents take system metrics as input, such as load, latency, CPU usage, memory usage, and the number of pods, and then provide scaling actions as output to control the number of Redis pods in the cluster, based on the results of a user defined reward function.

1) Dense Layer Agent - Modified DERP

The first agent employs a traditional dense layer architecture, as shown in the following network structure: We modify the original DERP [4] to improve its learning capacity and handle a broader set of system metrics. While the original DERP used throughput, latency, and the number of VMs as input, our modified version also incorporates CPU usage, memory usage, and the number of pods. This allows our agent to better capture the system's state and make more informed scaling decisions.

In addition, the original DERP used a single hidden layer in its architecture. To enhance the agent's learning capability, we introduced two hidden layers with more neurons. This Deeper architecture allows the agent to model more complex relationships between the input features and the system's scaling actions, leading to more efficient learning and improved performance in dynamic cloud environments.

- Input layer: The agent takes a state vector representing the current load, CPU usage, memory usage, latency, and the number of pods.
- Hidden layers: Two dense layers, each with 128 units, followed by ReLU activation functions to capture nonlinear relationships between the input features.
- Output layer: The output is the predicted Q-values corresponding to the actions that determine the appropriate scaling decision for the number of pods.

The dense layer agent's architecture is well-suited for situations where the input features (i.e., metrics) are independent and do not contain significant temporal patterns. The model is trained using a **Double DQN (Deep Q-Network)** algorithm [37] Figure 2, an enhancement over the traditional DQN algorithm [38]. In standard DQNs, the action selection and value estimation for Q-learning are performed by the same network, which can lead to overestimation of action values. This overestimation arises when the network incorrectly assigns high Q-values to suboptimal actions.

To mitigate this issue, **Double DQN** decouples the action selection and value estimation processes. Specifically, it uses two networks:

- **Online (Current) Network**: This network is used to select the best action for the given state.
- **Target Network**: This network is responsible for estimating the Q-value of the selected action.

By separating the action selection from the value estimation, Double DQN reduces the bias in Q-value estimation, leading to more stable and reliable learning. This improved approach allows the agent to better handle noisy environments and complex decision-making tasks, avoiding overly optimistic action-value predictions. Double DQN has been shown to significantly improve learning performance, especially in scenarios where there are many possible actions with varying rewards.

In this work, Double DQN (Deep Q Network) Figure 2 is employed to stabilize learning in the dense-layer-based agent, allowing it to make better reactive scaling decisions in the Kubernetes/Redis environment, particularly in the presence of dynamic and unpredictable workloads.

2) LSTM Agent: Handling Temporal Dependencies and Spikes - Resourcerer Supreme

The second agent, Resourcerer Supreme (RS), incorporates Long Short-Term Memory (LSTM) layers to capture the temporal dependencies in the workload, such as periodic spikes in user requests. This architecture enables the agent to go beyond traditional reactive scaling methods by anticipating future changes based on past data, allowing it to proactively scale resources.

In this approach, we also employ a Double DQN (Deep Q-Network) Figure 2 to further stabilize the learning process [37]. This combination of LSTM layers Figure 3 and DDQN enhances the RS agent's ability to adapt to both short-term fluctuations and long-term trends in system input load, ensuring more efficient resource utilization and lower latency.

- **Input layer:** Similar to the dense layer agent, the input is a state vector representing the current system metrics, but in the LSTM agent, the input is treated as a sequence of states, allowing the agent to capture time dependencies.
- **LSTM layers:** The agent consists of LSTM layers, which process these sequences of system metrics over time. The LSTM layers provide the agent with memory of past system behavior, helping it recognize patterns such as increasing or decreasing load trends.
- **Output layer:** The output remains the Q-values for scaling decisions, but the LSTM agent benefits from

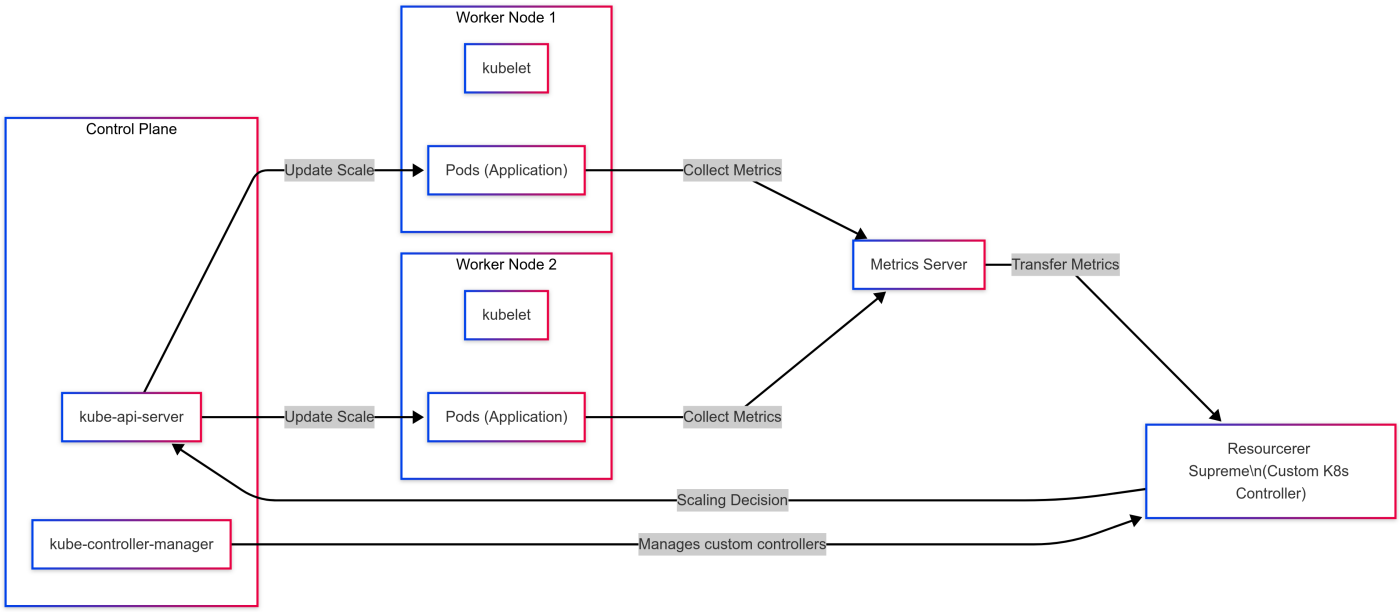


FIGURE 1: Simplified horizontal architecture and workflow of the Resourcerer Supreme (RS) autoscaling controller within a Kubernetes cluster. In this example, the cluster is depicted with two worker nodes, each hosting a kubelet and application Pods. Workload metrics generated by these Pods are collected by the Metrics Server and conveyed to RS. Acting as a custom Kubernetes controller, RS makes both proactive (predictive) and reactive scaling decisions—specifically, deciding whether to add or remove Pods—based on the received metrics. These scaling decisions are then communicated to the kube-api-server within the Control Plane, which updates the Deployment accordingly under the oversight of the controller manager.

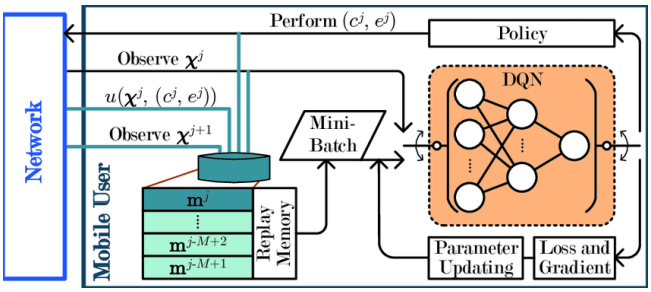


FIGURE 2: Double Deep Q-learning architecture that decouples action selection and value estimation. The online network selects actions based on current Q-value estimates, while a target network—updated periodically—provides stable Q-value targets for training.

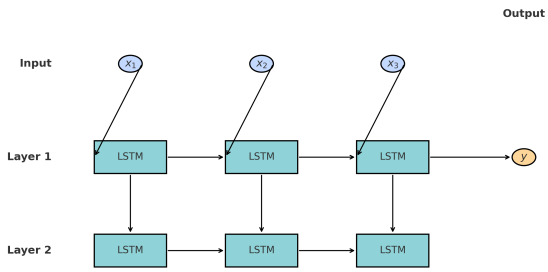


FIGURE 3: Illustrative example of an LSTM network architecture capturing temporal dependencies in system workloads. The stacked layers enhance the agent's ability to learn complex patterns over time. LSTM cells employ memory and gating mechanisms to retain long-term information and anticipate workload fluctuations.

its ability to consider the temporal patterns in the input data, making it more proactive than reactive.

3) RS with Transfer Learning for LSTM Agent

To further improve the adaptability of our LSTM-based agent (RS), we apply transfer learning by fine-tuning a pre-trained RS model, III-B2 on our specific Kubernetes/Redis environment. The agent begins with a model trained on general workload patterns and is then fine-tuned using data collected from our experimental setup. This approach allows the model to retain prior knowledge while adapting to the unique workload characteristics of our system.

In this work, we fine-tune the RS agent using an additional training phase with a smaller batch size to ensure efficient adaptation to the new environment. This fine-tuning phase allows the agent to refine its decision-making strategy based on real-world metrics while maintaining the generalization capabilities of the pre-trained model.

- **Input layer:** The input follows the same structure as the standard LSTM agent, representing system metrics as a

sequence of states.

- **LSTM layers:** The agent leverages pre-trained LSTM layers, which are fine-tuned with new workload-specific data, allowing better generalization to different deployment environments.
- **Output layer:** The output remains the Q-values for scaling decisions, with the fine-tuned model achieving improved responsiveness to workload fluctuations and spikes.

By transferring knowledge from previously trained workloads, the LSTM agent can generalize more effectively to unseen workload patterns with minimal retraining efforts. This significantly **reduces the training time** while improving the agent's ability to handle varying loads, periodic spikes, and latency fluctuations in real-world applications.

The final fine-tuned model outperforms both the baseline dense agent and the standard RS agent, demonstrating enhanced resource utilization, faster convergence, and increased responsiveness to workload variations. This method ensures that the system remains adaptable across different cloud environments and can be seamlessly deployed in diverse application scenarios, making it a robust and adaptable solution for real-time autoscaling challenges across various distributed environments and applications, beyond just Kubernetes-managed Redis deployments.

a: The Spike Factor:

It is important to mention that while the DERP agent can recognize periodicity patterns in workloads, it struggles to predict sharp spikes in demand. Many applications, such as those dealing with airplane ticket bookings around Christmas or dynamic pricing during high-demand periods, exhibit such spikes. Traditional scaling methods may fail to handle these abrupt changes effectively.

The LSTM based agents address this limitation by not only recognizing patterns but also anticipating these spikes. By analyzing historical trends, the agents proactively adjust resources to meet future demands. This makes our LSTM-based agents both reactive to current system states and proactive in scaling to meet future requirements, ensuring more efficient handling of sudden spikes in input load (see Figures 3), 4).

Deep RL Agent Deployment: All of our Deep RL-based autoscaling agents—including RS, Enhanced DERP, and RS Transfer—are deployed as custom Kubernetes controllers encapsulated within dedicated Pods. As illustrated in Figure 1 , these agents reside within the control plane and interact directly with the Metrics Server and the Kubernetes API server. They continuously receive workload metrics, make scaling decisions, and issue actions to the control loop. In the case of RS Transfer, pre-trained LSTM weights are loaded during initialization, enabling fast and efficient inference without relying on external model servers.

C. REWARD FUNCTION DESIGN

The reward function is crucial for guiding the Deep RL agents to optimize the system's performance. The function is designed to balance the competing objectives of maximizing the load handled by the Redis pods, while minimizing latency, CPU usage, memory usage, and the number of pods required to maintain the system. The reward function is formulated as:

$$\text{load_reward} = \text{load_factor} \times \text{next_load} \tag{1}$$

$$\text{latency_penalty} = \text{latency_factor} \times \text{next_latency} \tag{2}$$

$$\text{pods_penalty} = \text{pods_penalty_factor} \times \text{num_pods} \tag{3}$$

$$\text{cpu_penalty} = \text{cpu_penalty_factor} \times \text{next_cpu_usage} \tag{4}$$

$$\text{memory_penalty} = \text{memory_penalty_factor} \times \text{next_memory_usage} \tag{5}$$

$$\text{reward} = \text{load_reward} + \text{latency_penalty} \tag{6}$$

$$+ \text{pods_penalty} + \text{cpu_penalty} + \text{memory_penalty} \tag{7}$$

Where:

- **load_factor** rewards the system for handling more load.
- **latency_factor** penalizes increased latency to maintain lower response times.
- **pods_penalty_factor** discourages the use of excessive pods to optimize resource allocation.
- **cpu_penalty_factor** penalizes excessive CPU usage to promote efficiency.
- **memory_penalty_factor** penalizes excessive memory usage to ensure efficient use of memory resources.

This reward function serves as an example of how the Resourcerer Supreme (RS) agents can efficiently handle dynamic and user-defined reward functions. While we utilized this specific reward function to demonstrate the effectiveness of our approach, the RS agent's adaptability allows them to work with various reward formulations tailored to different optimization goals. Whether the focus is on maximizing throughput, minimizing latency, optimizing resource consumption, or any other user-defined criteria, the RS agent dynamically scales the system to meet these objectives, ensuring optimal performance while avoiding over-provisioning. The flexibility of our models highlights their robustness in various cloud environments, providing consistent and superior results in both simulation and real-world scenarios.

D. LOSS FUNCTION AND TRAINING

The agents are trained using the Huber loss function [39], which is known for its robustness in handling outliers in the predicted Q-values. The Huber loss combines the advantages of mean squared error (MSE) and mean absolute error (MAE), reducing sensitivity to outliers while ensuring convergence during training:

$$\mathcal{L}(x, y) = \begin{cases} 0.5(x-y)^2 & \text{for } |x-y| \leq \delta, \\ \delta \cdot (|x-y| - 0.5 \cdot \delta) & \text{otherwise} \end{cases} \tag{8}$$

Here, δ controls the transition between MSE and MAE, providing a smooth and stable learning process.

E. SIMULATION ENVIRONMENT

The agents are tested in a simulated Kubernetes environment, where Redis pods handle incoming user requests. The simulation environment replicates real-world scenarios by varying system load, CPU usage, memory usage, and latency. The input load follows a sinusoidal distribution with added noise, simulating traffic spikes that occur periodically. The key metrics—load, CPU usage, memory usage, and latency—are monitored over time.

a: Load Distribution:

The load applied to the Redis cluster is a function of time, modeled as a sinusoidal wave with added noise and a load spike at a specific time. The load distribution follows this formula:

$$\text{load}(t) = 50 \cdot \sin\left(\frac{2\pi t}{1000}\right) + 50 + 20 + \mathcal{N}(0, 0.2)$$

In this formula, the sinusoidal component represents periodic variations in the system load, while the noise term, modeled as a Gaussian distribution $\mathcal{N}(0, 0.2)$, adds random variations to simulate real-world unpredictability. Additionally, a load spike occurs between time 1500 and 1600, represented as:

$$\text{spike}(t) = \begin{cases} 80 \cdot \left|\sin\left(\frac{2\pi \cdot 10 \cdot (t-1500)}{100}\right)\right| & \text{if } 1500 \leq t < 1600 \\ 0 & \text{otherwise} \end{cases}$$

This models a sudden increase in load during the spike period, simulating scenarios such as flash sales or traffic surges.

b: CPU Usage:

The CPU usage is modeled as a uniform random variable within the range of 30% to 75%, and it may increase during the spike period:

$$\text{cpu_usage}(t) = U(30, 75) + \Delta_{\text{spike}}(t)$$

where $\Delta_{\text{spike}}(t)$ is an additional usage during the spike period, simulating increased processing demands.

c: Memory Usage:

Similarly, memory usage follows a uniform random distribution between 40% and 65%, with an additional increase during the spike:

$$\text{memory_usage}(t) = U(40, 65) + \Delta_{\text{spike}}(t)$$

This models fluctuating memory demands under regular load and increased demands during the spike.

d: Latency:

The latency also follows a sinusoidal pattern, with additional noise and a spike during the traffic surge:

$$\text{latency}(t) = 1.2 \cdot \sin\left(\frac{2\pi \cdot 2t}{1000}\right) + 1.2 + \mathcal{N}(0, 0.1) + \Delta_{\text{spike}}(t)$$

This models typical latency variations, with added spikes during the increased load period.

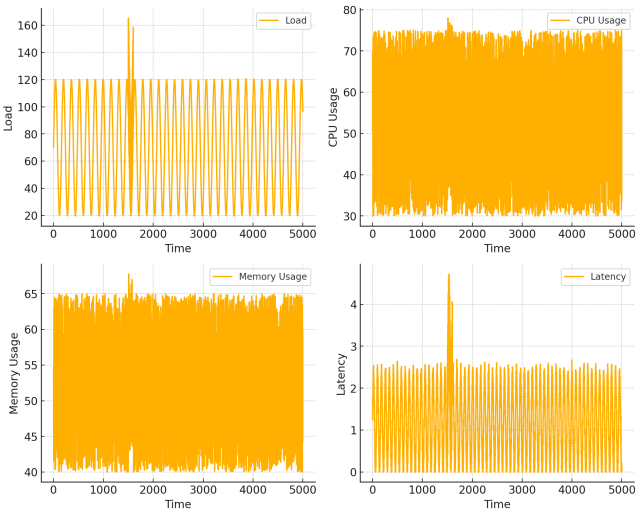


FIGURE 4: Simulation system metrics over time: Load, CPU Usage, Memory Usage, and Latency, with a sinusoidal pattern, noise, and a spike at time 1500.

The distributions chosen for CPU usage, memory usage, and latency were derived from empirical observations of real-world system behavior under similar load conditions. Our analysis indicated that CPU and memory utilization tend to vary within specific uniform ranges, with additional increases during load spikes, while latency exhibits a sinusoidal trend with superimposed noise to capture dynamic performance fluctuations. These choices ensure that the simulated environment closely mimics the actual performance characteristics observed during real deployments. A detailed discussion and analysis of these metrics, and how they react to the input load, is provided in Section V.

Our Deep RL agents interact with the Kubernetes metrics server, collecting real-time data on resource utilization and system performance. This information is fed into the reward function, and based on the calculated reward, the agent decides on the optimal number of Redis pods to scale in or out. The dense layer agent serves as a baseline, while the LSTM agents are expected to outperform it by capturing the temporal nature of workloads, resulting in more efficient scaling decisions.

F. RESOURCER SUPREME SCALING (RS) ALGORITHM

In our system, the Resourcer Supreme Scaling (RS) algorithm 1, powered by deep reinforcement learning, is responsible for dynamically adjusting the number of Redis pods in response to varying system metrics such as load, CPU usage, memory usage, and latency. This algorithm is employed for both the standard LSTM-based model and its enhanced variant that leverages transfer learning. The following algorithm outlines the key steps involved in the scaling decision process:

Algorithm 1 Resourcer Supreme Scaling (RS & RS Transfer) Algorithm

Require: Model type (**LSTM** or **LSTM Transfer**), Pre-trained Q-network (optional), Current system state: (load, cpu_usage, memory_usage, latency, num_pods)
Ensure: Optimal scaling action a^*

if Using **RS Transfer then**
  Load pre-trained $Q(s, a)$ from prior model
  Initialize adaptation rate α
else
  Initialize $Q(s, a)$ with random weights
end if
Initialize $\hat{Q}(s, a)$ (target network) with weights copied from $Q(s, a)$
Initialize replay buffer R
for each timestep t **do**
  Observe current state s_t
  if exploration condition is met **then**
    Select random action a_t
  else
    Select action $a_t = \arg\max_a Q(s_t, a)$
  end if
  Apply action a_t and observe next state s_{t+1} and reward r_t
  Store experience (s_t, a_t, r_t, s_{t+1}) in buffer R
  Sample minibatch from R
  for each experience in minibatch **do**
    Compute target $y = r_t + \gamma \max_a \hat{Q}(s_{t+1}, a)$
    if Using **RS Transfer then**
      Adapt weights using α:
      $Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha y$
    else
      Update $Q(s_t, a_t)$: $\mathcal{L} = (y - Q(s_t, a_t))^2$
    end if
  end for
  Periodically update $\hat{Q}$ from Q
  Set $s_t \leftarrow s_{t+1}$
end for
return optimal action a^* =0

a: Algorithm Explanation:

The RS algorithm operates as follows, with adaptations for both the standard RS (LSTM) model and RS Transfer:

- **Input:** At each timestep t, the system observes its current state, which includes the current load, CPU usage, memory usage, latency, and the number of active Redis pods.
- **Model Initialization:**
  - If using **RS (LSTM)**, the Q-network is initialized with random weights.
  - If using **RS Transfer**, a pre-trained Q-network is loaded to accelerate learning and adaptation.
- **Exploration vs. Exploitation:** Based on an exploration policy (e.g., ϵ-greedy), the agent either selects a random action (exploration) or the best-known action (exploitation) from the action-value function $Q(s, a)$.
- **Actions:** The possible actions are: `scale_in` (reduce the number of pods), `scale_out` (increase the number of pods), or `do_nothing` (maintain the current number of pods). In our system, both `scale_in` and `scale_out` modify the pod count by exactly one per decision. This design choice is based on experimental observations in our Kubernetes-Redis environment, where attempting to add or remove more than one pod in a single timestep led to instability. Specifically, rapidly scaling multiple pods at once disrupted the Redis cluster's rebalancing process and triggered connectivity issues, thereby compromising overall system performance.
- **Reward and Update:** The agent receives a reward based on the impact of its action, optimizing for resource efficiency and performance. The Q-values are updated as follows:
  - For **RS (LSTM)**, the update follows the standard Q-learning loss function.
  - For **RS Transfer**, an adaptation rate α is used to adjust Q-values, ensuring smooth knowledge transfer from the pre-trained model.
- **Replay Buffer and Stability:** A replay buffer is used to store experiences, and a separate target network is periodically updated to stabilize training.
- **Output:** The optimal action is determined based on the learned action-value function, allowing the system to scale resources efficiently in response to workload fluctuations.

IV. IMPLEMENTATION

A. TRAINING THE DEEP RL AGENTS

The deep reinforcement learning (RL) agents were trained using system metrics collected from the Kubernetes cluster. The input features for the agents were:

- Load (write/read requests to Redis),
- CPU usage percentage,
- Memory usage percentage,
- Latency
- Number of pods.

The environment was designed to simulate dynamic workloads with periodic spikes in system load, mimicking real-world usage patterns, see 4. The agent interacted with the environment by selecting actions (either adding, removing, or maintaining pods), receiving system state updates, and calculating rewards based on the following reward function:

$$\begin{aligned}
\text{load_reward} &= 3.0 \times \text{next_load}, \\
\text{latency_penalty} &= -5.0 \times \text{next_latency}, \\
\text{pods_penalty} &= -10.0 \times \text{num_pods}, \\
\text{cpu_penalty} &= -0.5 \times \text{next_cpu_usage}, \\
\text{memory_penalty} &= -0.5 \times \text{next_memory_usage}.
\end{aligned}$$

The cumulative reward (from each timestep) was used to guide the agents toward maximizing load handling while minimizing resource usage and latency. The agents were trained using a Double DQN (Deep Q-Network) architecture based on this reward function. It should be noted that the coefficients employed here (e.g., 3.0 for load, -5.0 for latency, -10.0 for pods, -0.5 for CPU and memory) are provided as an illustrative example of how a reward function can be formulated. In practice, any user- or vendor-designed reward function can be integrated into our system, allowing the agents to adjust and yield the desired outcomes tailored to specific performance and cost optimization objectives.

B. KUBERNETES AND REDIS CLUSTER CONFIGURATION

In our real-world experiments, the Redis cluster was configured within Kubernetes as a stateless service, with multiple Redis instances (pods) managed by Kubernetes. Each pod acted independently, and the autoscaling mechanism was originally controlled by the Kubernetes Horizontal Pod Autoscaler (HPA). For our experimentation, the HPA was disabled, and the Deep RL agents took control of the scaling decisions. This setup allowed us to simulate a fully dynamic environment where Redis pods were added or removed based on real-time system metrics.

The system was designed to handle varying input loads with spikes, allowing the Deep RL agents to learn optimal scaling strategies under changing conditions. The simulated environment generated dynamic workloads with oscillating and spiking patterns in load, CPU usage, memory usage, and latency. This ensured that the Deep RL agents were exposed to a range of scenarios, preparing them for real-world usage patterns.

C. MODIFICATIONS TO THE KUBERNETES ENVIRONMENT

The key modification made to the Kubernetes environment was the disabling of the HPA and the introduction of our custom Deep RL-based autoscaling agents. the Deep RL agents collected system metrics directly from the Kubernetes Metrics Server every 15 seconds, using these inputs to calculate rewards and make scaling decisions.

To ensure proper integration of the Deep RL agents into the Kubernetes environment, we modified the environment to allow direct control over the number of Redis pods. This involved creating a custom environment class that interfaced with Kubernetes through its API to dynamically increase or decrease the number of pods based on the agent's actions.

D. TECHNICAL CHALLENGES

During implementation, several technical challenges were encountered:

- **Handling latency**: Accurately capturing latency in real-time was critical for evaluating the performance of the Deep RL agents. We implemented a function that simulated latency spikes during periods of increased load to ensure that the agents learned how to react to sudden surges in demand.
- **Dynamic workloads**: The oscillating and spiking load patterns generated by the environment posed challenges for the agents. To ensure robustness, the agents were trained with both dense and LSTM architectures. The LSTM-based agents, in particular, were designed to capture the temporal patterns in workload changes, helping it anticipate future spikes and scale the system proactively.
- **Pods penalties**: Since adding more pods incurs costs, the reward function included a penalty for each additional pod. Striking the right balance between maximizing load handling and minimizing pods usage was a key challenge, and it required tuning the hyperparameters in the reward function.

Despite these challenges, the agents were successfully trained to autoscale the Redis pods, balancing load, latency, and resource utilization. The evaluation of the agents showed that the LSTM-based agents, which could capture temporal patterns, outperformed the dense layer agent by reacting more intelligently to workload spikes. see Section V-C.

V. EXPERIMENTAL RESULTS

A. EXPERIMENTAL SETUP

The experiments (simulations and real-world) were conducted on a multi-core Kubernetes environment (hosted on a large Ubuntu 20.04 VM), effectively acting as a cluster with the following specifications:

- **Memory:** 191 GiB of RAM,
- **CPUs:** 32 virtual CPUs, each based on the Intel Core Processor (Skylake) architecture, operating at 2 GHz,
- **GPU:** Cirrus Logic GD 5446 VGA compatible controller,
- **Storage:** 750 GiB available, formatted as EXT4.

The software environment included Python 3.8 with PyTorch for Deep learning, Kubernetes for managing Redis instances, and Matplotlib for visualizations. The cluster's key metrics—load, CPU usage, memory usage, latency, and number of pods (pods)—were monitored and used for the evaluation.

B. COMPARISON BETWEEN METHODS

We compared four methods of autoscaling the Redis cluster:

1) **Horizontal Pod Autoscaler (HPA):** The default autoscaler provided by Kubernetes, which scales the pods based on CPU utilization thresholds.
2) **Deep Reinforcement Learning with Dense Layers (RL Dense-DERP):** A Deep reinforcement learning (RL) agent using fully connected dense layers for decision-making.
3) **Deep Reinforcement Learning with LSTM Layers (RL LSTM-Resourcerer Supreme):** A Deep RL agent using LSTM layers to capture temporal patterns in the workload for more intelligent scaling decisions.

4) **Deep Reinforcement Learning with Transfer Learning (RL LSTM-Resourcerer Supreme Transfer):** An enhanced version of RL LSTM-Resourcerer Supreme that leverages transfer learning to quickly adapt to new workloads and application environments without retraining from scratch.

C. QUANTITATIVE RESULTS

The figures 6, 7, 8, 9 show a comparison of key metrics—CPU usage, memory usage, input load, read percentage, latency, and number of pods—over time for the best-performing agents (agents trained on different number of training steps) of HPA, DERP, Resourcerer Supreme (RS) and RS Transfer. The individual plots illustrate how each agent manages resources and responds to workload variations, while the combined figure provides an overall view. **(see Figures 5, 12 for the accumulated reward comparisons among HPA, DERP, RS, and RS Transfer).**

Figure 6 illustrates the suboptimal behavior of HPA under spiky loads, highlighting poor adaptive scaling. **Figure 7** shows how DERP, despite improvements, still lags behind RS in adaptive scaling during sudden surges.

Figure 8 highlights how the LSTM-based RS agent preemptively allocates pods when spikes are predicted, whereas **Figure 9** confirms the additional benefits of transfer learning, boosting final rewards beyond those of the baseline RS.

Figure 10 depicts the dramatic improvement RS Transfer achieves compared to RS in a real Kubernetes/Redis environment, while **Figure 11** provides a closer look at how proactive scaling decisions manifest in actual load, CPU, memory, and latency trends.

Simulation Results

In the simulation environment, agents faced periodic workload fluctuations and sudden traffic spikes. Accumulated rewards for each autoscaling approach were as follows:

- **HPA:** Achieved a negative reward of -1.01×10^6, indicating severe inefficiency primarily from over-provisioning due to its reactive scaling.
- **DERP:** Obtained approximately 2.2×10^5, improving upon HPA but still falling short of the advanced LSTM-based agents.
- **RS (LSTM):** Reached an accumulated reward of around 3.1×10^5, significantly outperforming both HPA and DERP by proactively scaling resources.
- **RS Transfer (Pre-trained & Fine-Tuned):** Achieved approximately 3.8×10^6, marking a dramatic improvement—12.26× higher than RS, clearly demonstrating the effectiveness of transfer learning in adapting rapidly and accurately to dynamic conditions.

Real-World Experiment Results

Real-world experiments provided insights into practical effectiveness when dealing with actual Kubernetes clusters hosting Redis pods under real traffic conditions, see Figures 11 10, 12:

- **HPA:** Produced a negative reward (-821), reflecting inefficient management of resources and high penalties due to over-provisioning and latency issues.
- **RS (LSTM):** Significantly improved performance, achieving a total reward of 5.6×10^4, marking a 6,921% improvement over HPA.
- **RS Transfer (Pre-trained & Fine-Tuned):** Further enhanced results, achieving 3.08×10^5 reward—5.5× greater than RS—demonstrating substantial benefits from transfer learning in real-world deployments.

We note that the real-world evaluation focuses on the RS and RS Transfer agents, as these two significantly outperformed Enhanced DERP during the simulation experiments. As a result, Enhanced DERP was not selected for deployment in the real-world setup. The Horizontal Pod Autoscaler (HPA), however, was tested alongside RS and RS Transfer in the real-world experiments to serve as a standard baseline. While HPA's performance was recorded, it was excluded from the main comparative plots due to its consistently poor scalability and unstable behavior under high-load conditions. Nevertheless, for completeness, its results are included in Figure 12 , which presents a tabular comparison of accumulated rewards and system-level metrics across all autoscalers in both simulation and real-world settings.

Cost Efficiency

Evaluating cost efficiency in terms of resource utilization and scaling accuracy, the RS agent notably outperformed traditional methods:

- RS improved cost efficiency by 143.7% over HPA and 40.9% over DERP in the simulation environment.
- RS Transfer further elevated these gains, achieving a 12.26× improvement over baseline approaches in simulation scenarios, effectively minimizing resource waste and ensuring optimal allocation.

Performance and Responsiveness

The advanced capabilities of RS and RS Transfer were clearly evident through their ability to proactively scale:

- The RS agent, enhanced by LSTM layers, captured temporal dependencies and anticipated load spikes, achieving rapid reward convergence and significantly better responsiveness compared to DERP and HPA.
- RS Transfer leveraged pre-trained knowledge, enabling faster adaptation and superior responsiveness—up to 12.26× better performance in simulations and 5.5× greater efficiency in practical deployments compared to RS.

Observations and Discussion

Why HPA Performs Poorly: HPA's negative rewards are primarily due to its reactive approach, exclusively relying on CPU and memory metrics. This often results in sustained over-provisioning of resources, leading to unnecessary cost increases without proportional gains in performance or throughput.

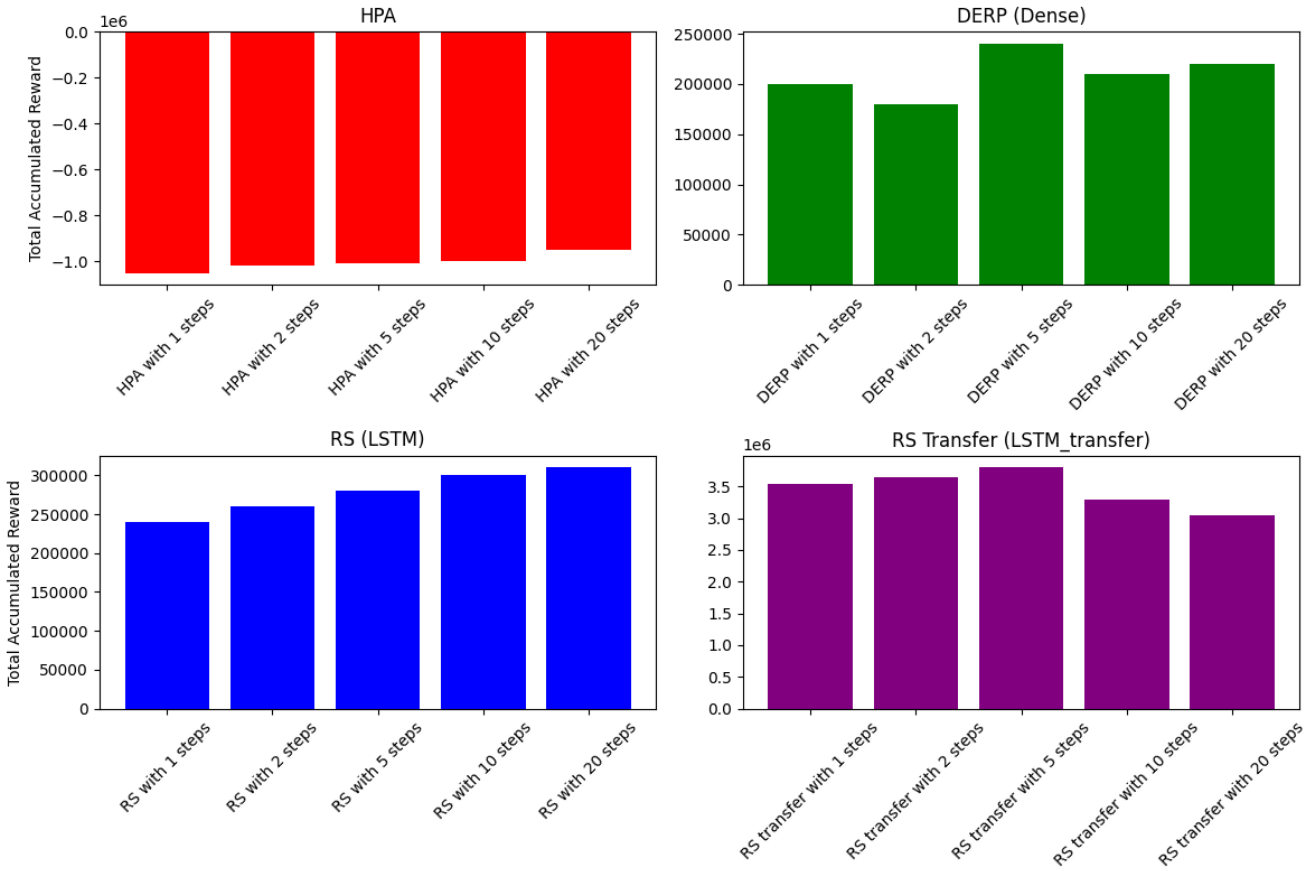


FIGURE 5: Final accumulated rewards for each autoscaler (HPA, DERP, RS, and RS Transfer) at different training steps (1, 2, 5, 10, and 20). RS Transfer (LSTM + Transfer Learning) achieves the highest rewards, surpassing 3.0×10^6 in most cases.

Rapid RS Convergence: The quick accumulation of high rewards by RS demonstrates that the model rapidly learned workload patterns, effectively predicting resource needs. Additional fine-tuning could further generalize and improve results across diverse workload scenarios.

Real-world Effectiveness of RS and RS Transfer: Remarkably, despite being trained primarily in simulated scenarios, RS and particularly RS Transfer have shown robust performance in real-world Kubernetes/Redis deployments. The substantial improvements in reward reflect optimized resource allocation, reduced latency, and more effective scaling strategies under dynamic, real-world conditions.

Overall, these results strongly support the integration of advanced Deep RL methods—particularly LSTM forecasting and transfer learning—into autoscaling mechanisms for cloud-native environments. These techniques ensure resource efficiency, lower operational costs, and reliable performance even under complex and unpredictable workload patterns.

D. TECHNICAL INSIGHTS

The experiments highlighted the following key insights:

- **Load Prediction:** The LSTM-based agents was better equipped to predict workload changes due to its temporal memory, leading to more stable scaling decisions. RS Transfer further enhances this by leveraging pre-trained knowledge, adapting quickly to new workload patterns without extensive retraining.
- **Latency Handling:** The Deep RL agents, particularly the LSTM-based, reduce latency more effectively by adjusting the number of pods in anticipation of future load increases, whereas the HPA often reacts too late. RS Transfer improves upon this by transferring learned scaling behaviors from previous environments, allowing for even faster and more efficient scaling responses.
- **Resource Utilization:** The LSTM-based agents keeps CPU and memory usage near optimal while minimizing pods. The RS Transfer variant benefits further from pre-trained knowledge, enabling it to generalize across different environments and workload conditions with reduced training overhead.

These updated results confirm the effectiveness of Deep RL-based autoscaling mechanisms, particularly the LSTM-based approaches, for dynamic and unpredictable workloads in Kubernetes/Redis. Leveraging transfer learning (RS Transfer) pushes performance even higher, demonstrating robust adaptability across diverse deployment scenarios, making it

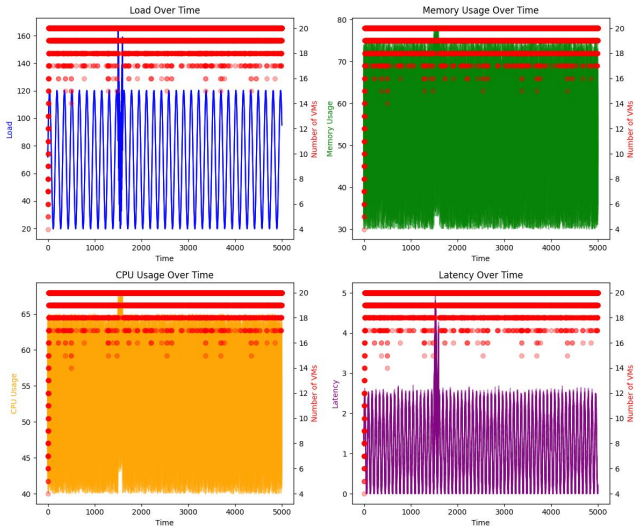


FIGURE 6: Performance metrics for the best-performing HPA agent. Despite training for 20,000 steps, HPA yields highly negative rewards (-1.01×10^6), struggling with sharp load spikes. This figure shows HPA's inability to adequately adapt to incoming load and other metrics by failing to scale the number of pods appropriately.

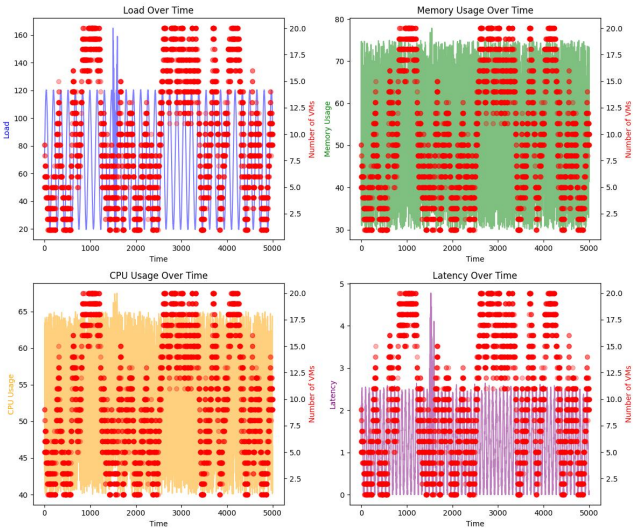


FIGURE 7: Performance metrics for the best-performing DERP agent. Trained for 10,000 steps, DERP achieves $\approx 2.2 \times 10^5$ reward, outperforming HPA but still struggling to adapt effectively to sharp load spikes, as indicated by suboptimal pod scaling decisions.

a viable solution beyond just Kubernetes-based Redis environments.

Finally, Figure 12 provides a concise summary of key performance metrics, clearly illustrating how each autoscaling method compares in terms of accumulated rewards, CPU and memory utilization, and average latency. Notably, RS Transfer outperforms all other approaches, both in simulation and real-world scenarios, underscoring its effectiveness in proactively adapting resources to meet workload demands while maintaining minimal latency and optimal resource consumption.

VI. DISCUSSION

A. ANALYSIS OF RESULTS

The results clearly demonstrate that the Deep reinforcement learning (RL) agents outperform the Kubernetes Horizontal Pod Autoscaler (HPA). This improvement can be attributed to the fact that Deep RL agents, especially the LSTM-based agents, learn to anticipate future states based on historical data, enabling more intelligent and proactive scaling decisions.

The HPA relies solely on current CPU and memory utilization, making it reactive and prone to late adjustments when workloads spike unexpectedly. In contrast, the Deep RL agents use a combination of metrics—load, CPU usage, memory usage, latency, and number of pods—and optimize the reward function to maximize overall system performance. This makes the Deep RL agents capable of handling dynamic workloads more efficiently, leading to significant cost savings and better performance in terms of latency and resource usage.

B. FUTURE DIRECTIONS AND POTENTIAL ENHANCEMENTS

While the results are highly encouraging, there remain a few considerations for further exploration:

- **Scalability:** Our Deep RL agents (dense and LSTM-based) have demonstrated strong performance on a Redis cluster, consistently surpassing HPA. While future work may explore deployment on even larger clusters, we expect **Resourcerer Supreme (RS)** agents to continue adapting effectively, leveraging their learning-based approach. Moreover, **RS Transfer** further enhances scalability by transferring learned behaviors, reducing the need for extensive retraining and accelerating adaptation to new environments.
- **Training Efficiency:** Training Deep RL agents, particularly the LSTM-based variant (RS without transfer learning), requires computational resources. However, the long-term benefits of improved scalability and resource efficiency far outweigh these costs. Additionally, **RS Transfer** significantly reduces training overhead by leveraging pre-trained models, enabling rapid convergence and seamless adaptation to new workload patterns with minimal fine-tuning.

C. GENERALIZATION AND LIMITATIONS

Our Deep RL agents were trained and tested in a Kubernetes/Redis environment, yet their adaptable design makes them well-suited for broader applications.

- **Flexible Optimization:** A key strength of our approach is its ability to adjust to different reward structures. Whether prioritizing cost reduction, latency minimiza-

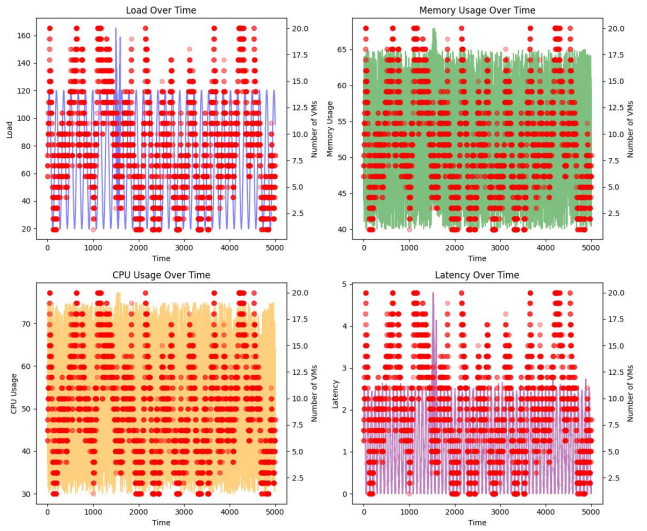


FIGURE 8: Performance of the best RS (LSTM) agent. Trained with 10,000 steps, RS achieves $\approx 3.1 \times 10^5$ accumulated reward, proactively handling workload spikes by adaptively scaling the number of pods according to incoming load and other metrics.

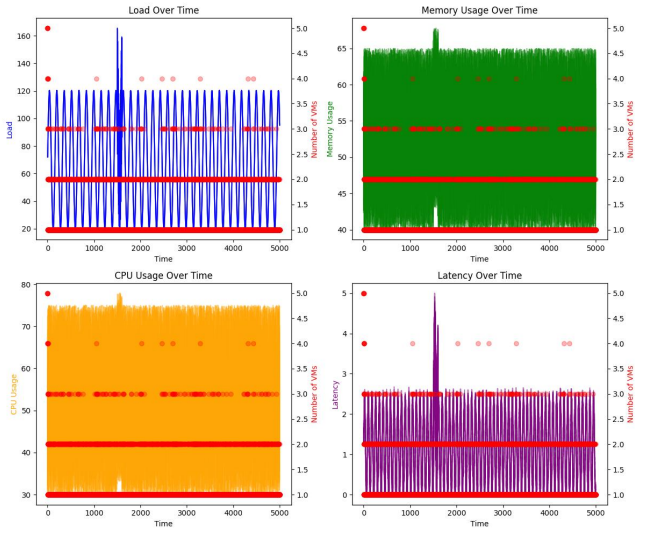


FIGURE 9: Performance of RS Transfer. Leveraging pre-trained knowledge, RS Transfer achieves superior scalability, effectively adapting to dynamic workload conditions by scaling the number of pods, thus reducing latency and optimizing resource utilization better than RS alone.

tion, or resource efficiency, the Deep RL agents can be fine-tuned for diverse cloud-native and stateful applications with minimal modifications.

- **Seamless Cloud Integration:** While tested in an on-premise Kubernetes setup, our method is designed to work across cloud platforms such as AWS, Google Cloud, and Azure. By adjusting the reward function to factor in cloud-specific parameters—such as billing models and network overhead—the agent can maintain optimal performance and efficiency across different infrastructures.
- **Training and Convergence:** The enhanced DERP agent, based on dense layers, achieves satisfactory training times and convergence behavior. However, the LSTM-based RS agent required significantly longer training periods and faced convergence challenges. To address this, we proposed RS Transfer—a transfer learning variant—which dramatically reduced training time and yielded the best performance across metrics.
- **Adaptability to Spikes and Novel Patterns:** Both the RS and RS Transfer agents demonstrate strong adaptability to workload spikes and previously unseen input distributions. This is attributed to the architectural choice of using LSTMs, which effectively model temporal dependencies, and the reinforcement learning paradigm itself, which enables learning in dynamic environments. Our agents can generalize to new vendor-defined reward structures, load patterns, and deployment configurations with minimal retraining.

Beyond generalization, our system demonstrates strong scalability characteristics. The synthetic workload used in our experiments includes high-variance noise and sudden, large spikes—conditions that stress-test system responsiveness. The fact that both RS and RS Transfer maintain low latency and stability under such stress validates their adaptability to dynamic workloads. Furthermore, the scalability of our approach is inherently tied to the scalability of the underlying orchestration platform—i.e., Kubernetes. Our agent introduces no architectural bottleneck of its own and remains functional as long as the cluster and the metrics infrastructure (e.g., Metrics Server) scale accordingly. Thus, the only scaling limitations stem from Kubernetes or the specific deployment environment, not from the agent itself.

Future work should explore distributed coordination techniques to overcome potential bottlenecks of centralized control, as well as asynchronous metric pipelines to mitigate the impact of stale observations. Broader evaluations on heterogeneous cloud environments and multi-tenant clusters will further validate the robustness of our approach. Additionally, exploring hybrid approaches, such as combining RL with traditional autoscalers, might mitigate some of the limitations identified in this study.

VII. CONCLUSION AND FUTURE WORK

A. CONCLUSION

This paper presented a Deep Reinforcement Learning (RL) approach to autoscaling Redis clusters in Kubernetes, significantly improving scalability and resource efficiency. Our Deep RL agents, utilizing both dense and LSTM layers, outperformed the default Kubernetes Horizontal Pod Autoscaler (HPA) by a wide margin. In simulations, our LSTM-based agent (**RS**) achieved a 207% increase in cost efficiency over HPA and a 40%performance improvement over the dense Deep RL agent. Additionally, our transfer learning variant (**RS Transfer**) leveraged pre-trained knowledge to improve

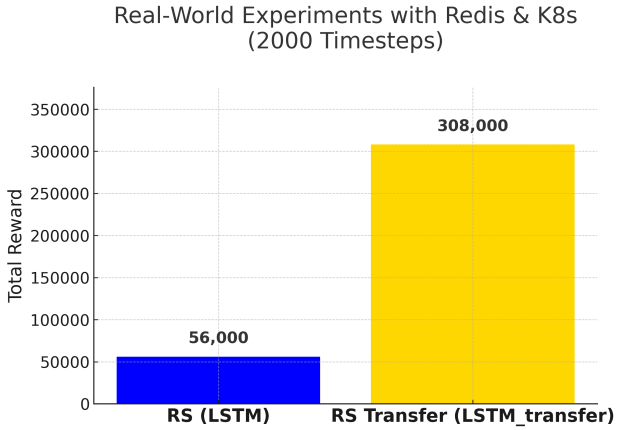


FIGURE 10: Real-world reward comparison of RS vs. RS Transfer over 2000 timesteps. RS Transfer achieves a 5.5× improvement, with a total reward of 308,000.

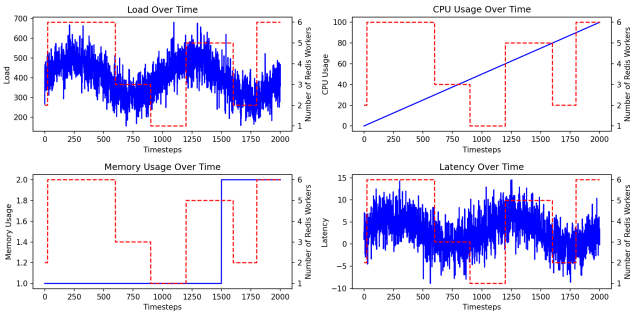


FIGURE 11: Detailed real-world workload analysis for RS Transfer. It dynamically adjusts Redis workers to minimize latency and optimize resources.

Method	Environment	Accumulated Reward
Horizontal Pod Autoscaler (HPA)	Simulation	-1.01×10^6
DERP	Simulation	2.2×10^5
Resourcerer Supreme (RS)	Simulation	3.1×10^5
RS Transfer	Simulation	3.8×10^6
Horizontal Pod Autoscaler (HPA)	Real-world	-821
Resourcerer Supreme (RS)	Real-world	5.6×10^4
RS Transfer	Real-world	3.08×10^5

FIGURE 12: Comparative summary of accumulated rewards in simulation and real-world experiments. RS Transfer consistently achieves the highest accumulated reward, highlighting its adaptability and resource efficiency.

upon RS by up to 12× in simulations and 5.5× in real-world tests, demonstrating superior adaptability and generalization.

The key contributions of this work lie in leveraging Deep RL to enable proactive and intelligent autoscaling decisions based on load, latency, CPU usage, memory consumption, and pod count. By optimizing a reward function that holistically captures these factors, our Deep RL agents effectively minimized latency while improving resource utilization. Moreover, **RS Transfer** highlights the practical benefits of transfer learning in cloud environments, reducing training overhead while accelerating convergence across different workload patterns. These results confirm that Deep RL, when enhanced with LSTM architectures and transfer learning, offers a powerful solution for adaptive, cost-efficient, and high-performance autoscaling in cloud-native environments.

B. FUTURE WORK

There are several promising directions for future research to extend and refine the methods presented in this paper:

a: Stateful Set Applications with Cassandra

One important future direction is to apply our Deep RL-based autoscaling techniques to stateful applications, particularly in databases such as Cassandra. While Redis is stateless and operates in-memory, Cassandra, a distributed NoSQL database known for its high availability and scalability, presents different challenges due to its persistent data storage and stateful nature [40]. In a StatefulSet, maintaining consistent state across pods when scaling up or down is crucial. This introduces complexities that require Deep RL agents to consider not only workload patterns but also data replication and consistency across nodes. By exploring these challenges, we aim to extend the versatility of Deep RL autoscalers to handle a wider range of applications, with a focus on databases where data persistence and state maintenance are critical. The **RS Transfer** approach could be particularly useful here, allowing previously learned scaling policies from Redis deployments to bootstrap training for Cassandra clusters.

b: Incorporating Additional Metrics

Currently, our Deep RL agents make scaling decisions based on load, CPU usage, memory usage, and latency. Future work will aim to incorporate additional metrics such as disk I/O, network bandwidth, and storage capacity, particularly when applying the model to stateful services. These metrics could provide a more holistic view of system health and further optimize resource utilization. Moreover, **RS Transfer** can help adapt to these additional metrics more quickly, as existing models can be fine-tuned rather than trained from scratch.

c: Exploring Other Deep RL Architectures

Our study focused on the use of Double Deep Q-Networks (DDQN) with dense and LSTM layers. However, there are other advanced Deep RL architectures that could be explored to improve performance, including Dueling Deep Q-Networks (Dueling DQN) [41], which separates the value and advantage streams, potentially providing more efficient

learning. Additionally, other architectures like Proximal Policy Optimization (PPO) [42] or Soft Actor-Critic (SAC) [43] could be explored for continuous action spaces, especially in scenarios with more granular scaling actions, such as adjusting specific pod configurations or network bandwidth. **RS Transfer** could also be applied across these alternative algorithms, transferring learned policies to new models.

C. SUMMARY OF KEY FINDINGS

In this work we introduced a Deep Reinforcement Learning (Deep RL) approach for autoscaling Redis clusters in Kubernetes, demonstrating significant advantages over both the default Horizontal Pod Autoscaler (HPA) and the modified DERP agent. Our Deep RL agents, due to their the LSTM-based nature, **Resourcerer Supreme (RS), RS Transfer**, effectively learned workload patterns and proactively adjusted resources, leading to superior cost efficiency, lower latency, and better overall resource utilization.

In simulation environments, RS (LSTM) achieved an accumulated reward of approximately 3.1×10^5, compared to DERP's 2.2×10^5 and HPA's -1.01×10^6, clearly illustrating its ability to capture temporal dependencies and proactively scale resources. Moreover, RS Transfer (Pre-trained & Fine-Tuned) dramatically outperformed RS by achieving an accumulated reward of about 3.8×10^6 in simulations—roughly 12.26× higher—and delivered 5.5× better performance in real-world tests.

These results reinforce the benefits of Deep RL-powered predictive autoscaling, which dynamically optimizes resource allocation based on learned workload behaviors rather than reacting solely to CPU or memory utilization. **RS Transfer**, in particular, demonstrates the power of transfer learning in cloud environments, significantly reducing training overhead while improving adaptability across different workloads and system configurations.

Beyond Kubernetes/Redis, RS Transfer offers an important advantage: rapid adaptation to new environments and applications with minimal retraining. By leveraging knowledge from prior deployments, it enables seamless transfer of scaling intelligence to diverse cloud-native architectures, paving the way for broader applications of Deep RL-based autoscaling in complex, distributed systems.

REFERENCES

[1] Y.-J. Yeom, T. Kim, D.-H. Park, and S. Kim, "Horizontal pod autoscaling in Kubernetes for elastic container orchestration," Sensors, vol. 20, no. 16, p. 4621, 2020.

[2] S. Sanfilippo and M. Stancliff, "Redis: A high-performance, in-memory, key-value store," Redis Labs, 2013. [Online]. Available: https://redis.io/

[3] S. M. Thampi, "Introduction to distributed systems," CoRR, vol. abs/0911.4395, 2009. [Online]. Available: http://arxiv.org/abs/0911.4395

[4] C. Bitsakos, I. Konstantinou, and N. Koziris, "Derp: A Deep Reinforcement Learning cloud system for elastic resource provisioning," in 2018 IEEE International Conference on Cloud Computing Technology and Science (CloudCom). IEEE, 2018, pp. 21–29.

[5] K. Yu, Y. Liu, and Q. Wang, "Review of Deep Reinforcement Learning," IEEE Access, 2020. [Online]. Available: https://doi.org/10.1109/ACCESS.2020.2979650

[6] Y. Yu, X. Si, Z. Hu, and J. Zhang, "A review of recurrent neural networks: LSTM cells and network architectures," Neural Computation, 2019. [Online]. Available: https://doi.org/10.1162/neco_a_01199

[7] "Horizontal pod autoscaler," https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/, 2024, accessed: 2024-10-22.

[8] "Vertical pod autoscaler," https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#vertical-pod-autoscaling, 2024, accessed: 2024-10-22.

[9] "Cluster autoscaler," https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/README.md, 2024, accessed: 2024-10-22.

[10] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, no. 3, pp. 279–292, 1992.

[11] K. Lolos, I. Konstantinou, V. Kantere, and N. Koziris, "Elastic management of cloud applications using adaptive reinforcement learning," 2017 IEEE International Conference on Big Data (Big Data), pp. 203–212, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:19567764

[12] A. W. Services, "Auto scaling documentation," 2025, accessed: March 24, 2025. [Online]. Available: https://docs.aws.amazon.com/autoscaling/

[13] M. L. Puterman, Markov Decision Processes: Discrete Stochastic Dynamic Programming. New York, NY, USA: John Wiley Sons, Inc., 1994.

[14] D. Tsoumakos, I. Konstantinou, C. Boumpouka, S. Sioutas, and N. Koziris, "Automated, elastic resource provisioning for nosql clusters using tiramola," in 13th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid). IEEE, 2013, pp. 34–41.

[15] J. Rao, X. Bu, C. Xu, L. Wang, and G. Yin, "Vconf: A reinforcement learning approach to virtual machines auto-configuration," in Proceedings of the 6th International Conference on Autonomic Computing (ICAC '09). ACM, 2009, pp. 137–146.

[16] M. Imdoukh, I. Ahmad, and M. G. Alfailakawi, "Machine learning-based auto-scaling for containerized applications," Neural Computing and Applications, vol. 32, no. 13, pp. 9745–9760, 2020. [Online]. Available: https://doi.org/10.1007/s00521-019-04507-z

[17] L. Toka, G. Dobreff, B. Fodor, and B. Sonkoly, "Machine learning-based scaling management for kubernetes edge clusters," IEEE Transactions on Network and Service Management, vol. 18, no. 1, pp. 958–972, 2021.

[18] G. Koukis, S. Skaperas, I. A. Kapetanidou, L. Mamatas, and V. Tsaoussidis, "Performance evaluation of kubernetes networking approaches across constraint edge environments," 2024. [Online]. Available: https://arxiv.org/abs/2401.07674

[19] B. Thurgood and R. G. Lennon, "Cloud computing with Kubernetes cluster elastic scaling," ACM, 2019. [Online]. Available: https://dx.doi.org/10.1145/3341325.3341995

[20] S. Lankes, "Hermitcore: A unikernel for extreme scale computing," ACM, 2016. [Online]. Available: https://dx.doi.org/10.1145/2931088.2931093

[21] H. Kimm, Z. Li, and H. Kimm, "Scadis: Supporting reliable scalability in Redis replication on demand," IEEE, 2017. [Online]. Available: https://dx.doi.org/10.1109/SmartCloud.2017.9

[22] H. Zhu, Z. Bai, and J. Li, "Harmonia: Near-linear scalability for replicated storage with in-network conflict detection," ACM, 2019. [Online]. Available: https://dx.doi.org/10.14778/3368289.3368301

[23] N. Ben Seghier and O. Kazar, "Performance benchmarking and comparison of nosql databases: Redis vs mongodb vs cassandra using ycsb tool," IEEE, 2021. [Online]. Available: https://dx.doi.org/10.1109/ICRAMI52622.2021.9585956

[24] A. I. Sanka, M. Chowdhury, and R. Cheung, "Efficient high-performance fpga-Redis hybrid nosql caching system for blockchain scalability," Elsevier, 2021. [Online]. Available: https://dx.doi.org/10.1016/j.comcom.2021.01.017

[25] P. Li, B. Luo, W. Zhu, and H. Xu, "Cluster-based distributed dynamic cuckoo filter system for Redis," Taylor & Francis, 2019. [Online]. Available: https://dx.doi.org/10.1080/17445760.2019.1599889

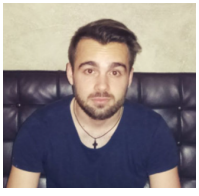
[26] M. Malhotra et al., "Dynamic scaling of web services for xen based virtual cloud environment," International Journal of Cloud Computing, 2020. [Online]. Available: https://doi.org/10.1504/IJCC.2020.10034234

[27] C. Ardagna et al., "A competitive scalability approach for cloud architectures," IEEE Transactions on Services Computing, 2014. [Online]. Available: https://doi.org/10.1109/TSC.2014.2372786

[28] O. Kuchaiev and B. Ginsburg, "Factorization tricks for LSTM networks," arXiv preprint arXiv:1703.10722, 2017. [Online]. Available: https://arxiv.org/abs/1703.10722

[29] J. Ikemoto et al., "Application of Deep Reinforcement Learning to control problems," in International Symposium on Control, Automation, and Systems, 2019. [Online]. Available: https://doi.org/10.23919/ICCAS.2019.8912116

[30] P. Liu et al., "Scanflow-k8s: Agent-based framework for autonomic management and supervision of ml workflows in Kubernetes clusters," in International Conference on Machine Learning and Applications, 2020. [Online]. Available: https://doi.org/10.1109/ICMLA.2020.00041
[31] M. Fogli et al., "Performance evaluation of Kubernetes distributions in federated cloud infrastructure," in IEEE International Conference on Cloud Computing, 2021. [Online]. Available: https://doi.org/10.1109/CLOUD.2021.00073
[32] S. J. Pan and Q. Yang, "A survey on transfer learning," IEEE Transactions on Knowledge and Data Engineering, vol. 22, no. 10, pp. 1345–1359, 2010.
[33] N.-M. Dang-Quang and M. Yoo, "Deep learning-based autoscaling using bidirectional long short-term memory for Kubernetes," Applied Sciences, vol. 11, no. 9, 2021. [Online]. Available: https://www.mdpi.com/2076-3417/11/9/3835
[34] "Prometheus: Monitoring system and time series database," https://prometheus.io, 2021, accessed: 2024-10-21.
[35] "Grafana: The open platform for analytics and monitoring," https://grafana.com, 2021, accessed: 2024-10-21.
[36] "Kubernetes metrics server," https://github.com/kubernetes-sigs/metrics-server, 2024, accessed: 2024-10-21.
[37] A. H. Bell-Thomas, "Exploring variational deep q networks," arXiv preprint arXiv:2004.05615, 2020. [Online]. Available: https://arxiv.org/abs/2004.05615
[38] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski et al., "Human-level control through Deep Reinforcement Learning," Nature, vol. 518, no. 7540, pp. 529–533, 2015.
[39] P. J. Huber, "Robust estimation of a location parameter," Annals of Mathematical Statistics, vol. 35, no. 1, pp. 73–101, 1964.
[40] A. Lakshman and P. Malik, "Cassandra: A decentralized structured storage system," ACM SIGOPS Operating Systems Review, vol. 44, no. 2, pp. 35–40, 2010.
[41] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for Deep Reinforcement Learning," in Proceedings of the 33rd International Conference on Machine Learning (ICML). JMLR.org, 2016, pp. 1995–2003.
[42] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, 2017.
[43] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy Deep Reinforcement Learning with a stochastic actor," in Proceedings of the 35th International Conference on Machine Learning (ICML). JMLR.org, 2018, pp. 1861–1870.



CONSTANTINOS BITSAKOS is a PhD candidate at the Computing Systems Laboratory (CSLab) of the National Technical University of Athens (NTUA) since 2018. He holds an M.Sc. in Big Data and Machine Learning from NTUA (2020–2022) and an M.Sc. in Electrical and Computer Engineering from NTUA (2008–2015). He is the former co-founder of the web company Filbit. His research interests include large distributed systems, scalability on distributed systems, deep learning, and quantum computing.



DIMITRIOS TSOUMAKOS is an Associate Professor of Big Data Management Systems at the School of Electrical and Computer Engineering of the National Technical University of Athens (NTUA), where he also serves as the Director of DBLab. He received his Diploma in Electrical and Computer Engineering from NTUA in 1999, and earned his M.Sc. and Ph.D. degrees in Computer Science from the University of Maryland in 2002 and 2006, respectively. His research interests focus on large-scale distributed systems, big data management, and algorithmic/optimization aspects.



IOANNIS KONSTANTINOU is an Associate Professor in the Department of Informatics and Telecommunications at the University of Thessaly, where he teaches courses in Operating Systems and Programming. He is also a senior researcher at CSLab, NTUA, and a member of the Board of Directors at KTP SA. He holds a Diploma (2004), an M.Sc. in Techno-Economic Systems (2007), and a PhD (2011) from NTUA. His research focuses on large-scale distributed data management systems. He is a recipient of a Best Paper Award (IEEE CCGRID 2013) and a Best Paper Nomination (IEEE CCGRID 2015).



NECTARIOS KOZIRIS is a Professor of Computer Science and former Dean of the School of Electrical and Computer Engineering at NTUA. His research spans parallel and distributed systems, hyperconverged data centers, scalable data management, and storage systems. He has published over 180 papers with more than 6600 citations (h-index: 36), and received best paper awards at IEEE/ACM IPDPS 2001 and CCGRID 2013. He co-founded the Greek Free/Open Source Software Society (GFOSS), served as Vice-Chair of GRNET (2004–2014), and leads national and EU-scale HPC initiatives such as okeanos and EuroHPC JU.